Next-best-product Recommendation

Gợi ý cụm khách sạn tiếp theo cho đặt phòng nghỉ dưỡng

Sử dụng bộ dữ liệu Expedia công khai

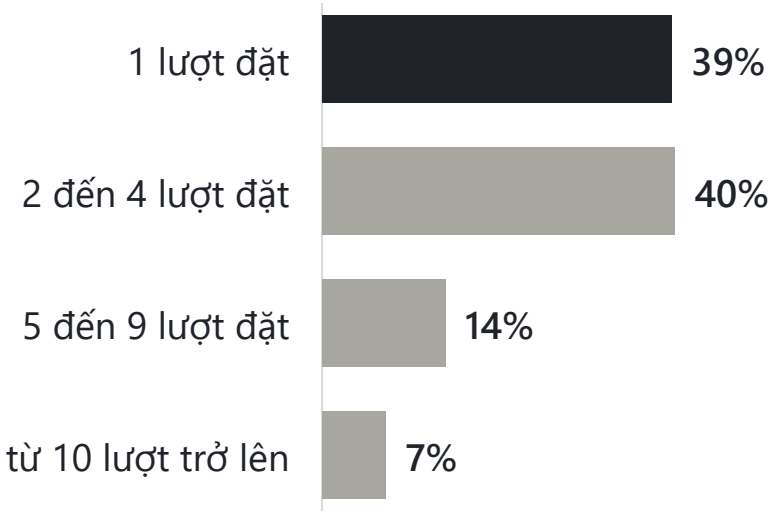
Expedia: 2,952,675 lượt đặt phòng, 100 cluster để xếp hạng

Đại lượng	Giá trị
Dòng thô trong file (click và đặt phòng)	37,670,293
Trong đó là click, không dùng làm nhãn	92.0%
Lượt đặt phòng sau làm sạch	2,952,675
Người dùng	813,985
Đối tượng cần xếp hạng (hotel_cluster)	100
Điểm đến khác nhau	36,933
Thời gian	2013-01-07 đến 2014-12-31
Người dùng chỉ đặt một lần	39.3%
Đoàn khách: solo, đôi, gia đình, nhóm	29.0%, 44.1%, 18.7%, 8.1%

Định nghĩa sau được tự suy ra từ số người lớn và trẻ em trong tìm kiếm: solo = 1 người lớn, đôi = 2 người lớn, nhóm = từ 3 người lớn, gia đình = có trẻ em (bất kể số người lớn)

Nguồn: results/week4_rebuild/bookings_counts.json, eda_clean.json (E2, E4); quy tắc nhóm khách: src/nbp/data/clean.py (party_type). Expedia, 2013 đến 2014, sau làm sạch.

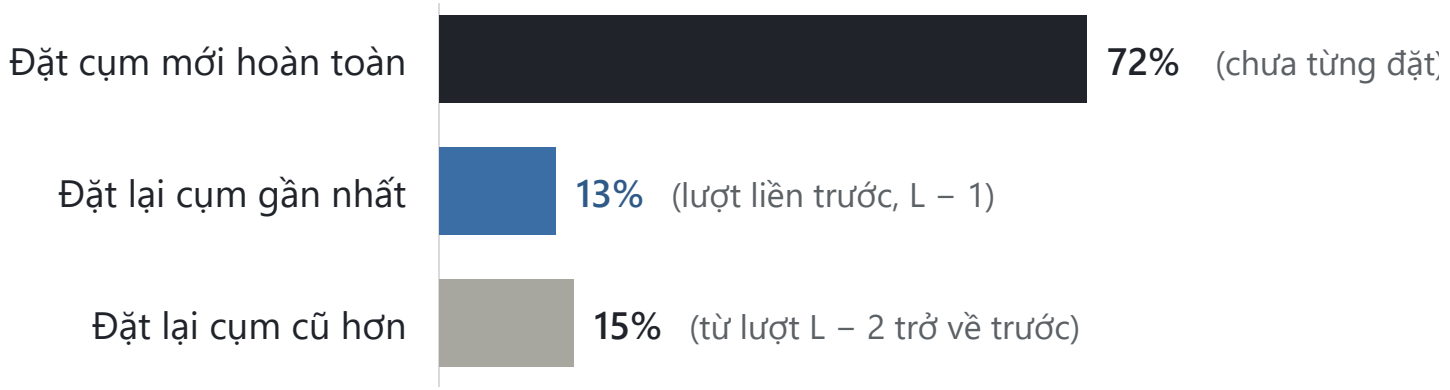
Expedia có 2,952,675 lượt đặt phòng của 813,985 người dùng, nhưng 39% chỉ đặt một lần



Trung vị là 2 lượt đặt mỗi người dùng.

Nguồn: results/week4_rebuild/eda_clean.json (E2), bookings_counts.json. Lượt đặt Expedia, mọi người dùng.

Khi có lịch sử, 72% lượt đặt là cụm mới; đặt lại cụm cũ chiếm 28% (13% là cụm gần nhất)



Tổng tỷ lệ đặt lại cụm cũ là 28%. Đa số khách (72%) chuyển sang cụm phòng hoặc gói mới trong chuyến đi tiếp theo.

Nguồn: data/interim/expedia_bookings.parquet. Tính trên 2,138,690 lượt đặt của khách đã có lịch sử trước đó (L ≥ 1).

Các hướng tiếp cận phổ biến của bài toán hệ gợi ý

Hướng tiếp cận	Ý tưởng chính
Content-based	Dựa trên thuộc tính của item
Collaborative filtering, matrix factorization	Dựa trên ma trận người dùng × item
Graph-based	Dựa trên đồ thị người dùng – item
Session-based	Dựa trên chuỗi hành vi trong một phiên
Sequential (SMLP4Rec)	**Dựa trên thứ tự các lượt đặt trong toàn bộ lịch sử**
Context-aware	Dựa trên ngữ cảnh của lần tìm kiếm

Mô hình hóa bài toán



Đầu ra là top k xếp hạng trong 100 hotel_cluster, đại diện (proxy) cho hạng phòng hoặc gói.

Mọi mô hình train và chấm trên toàn bộ lượt đặt, chia theo thời gian

2013-01-07 – 2014-10-08	2014-10-08 – 2014-11-19	2014-11-19 – 2014-12-31
train	**valid**	**test**
2,424,706 lượt đặt trong đó warm 1,710,952 và đầu tiên 713,754	264,638 lượt đặt trong đó warm 213,869 và đầu tiên 50,769	263,331 lượt đặt trong đó warm 213,869 và đầu tiên 49,462

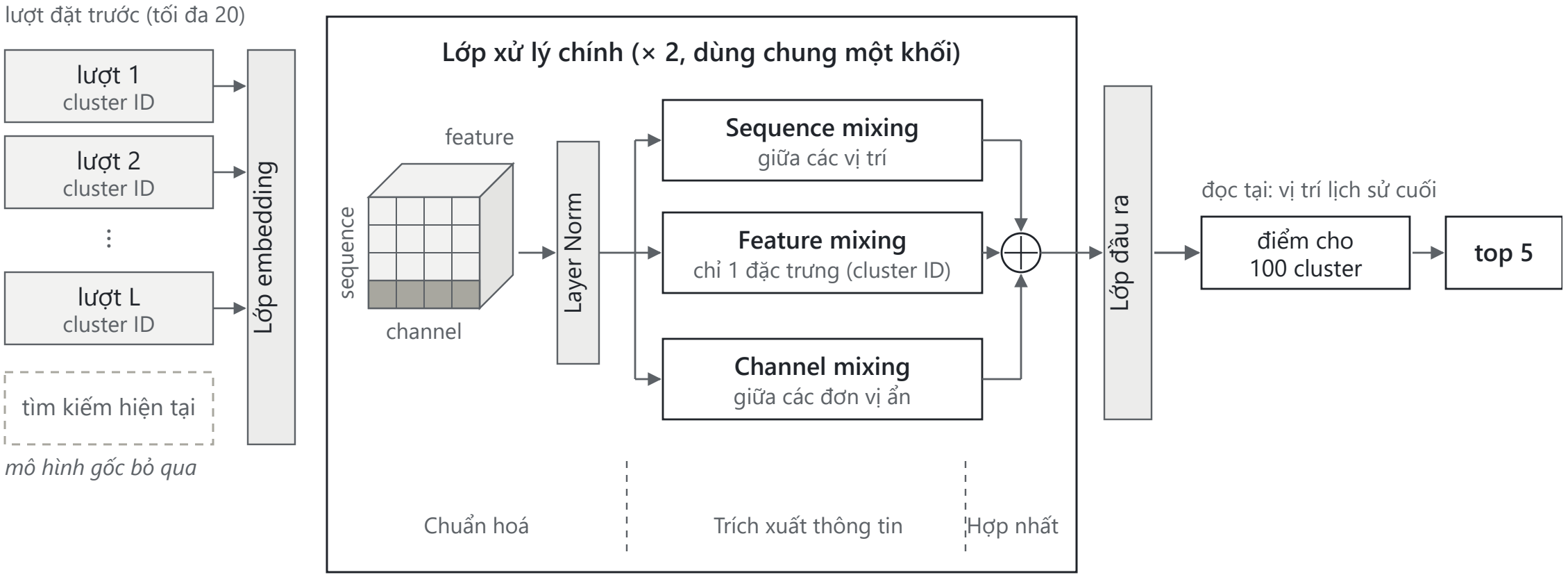
Train, valid và test đều gồm mọi lượt đặt; warm / đầu tiên chỉ là hai phần để báo cáo riêng

warm = đã có ít nhất một lượt đặt trước (L ≥ 1); đầu tiên (cold) = lịch sử rỗng (L = 0). Riêng SMLP4Rec gốc có thêm bản chỉ train trên warm

Xếp hạng đầy đủ trên cả 100 cluster. Recall và NDCG tại 5, 10 và 20. Cấu hình chọn trên valid, test chỉ đọc một lần.

Nguồn: results/week4_rebuild/eda_clean.json (splits), split_export_counts.json.

SMLP4Rec gốc chỉ đọc các lượt đặt trước đó và bỏ qua tìm kiếm hiện tại



Chỉ dùng lịch sử, SMLP4Rec gốc hơn ItemKNN một chút và thua xa logistic regression

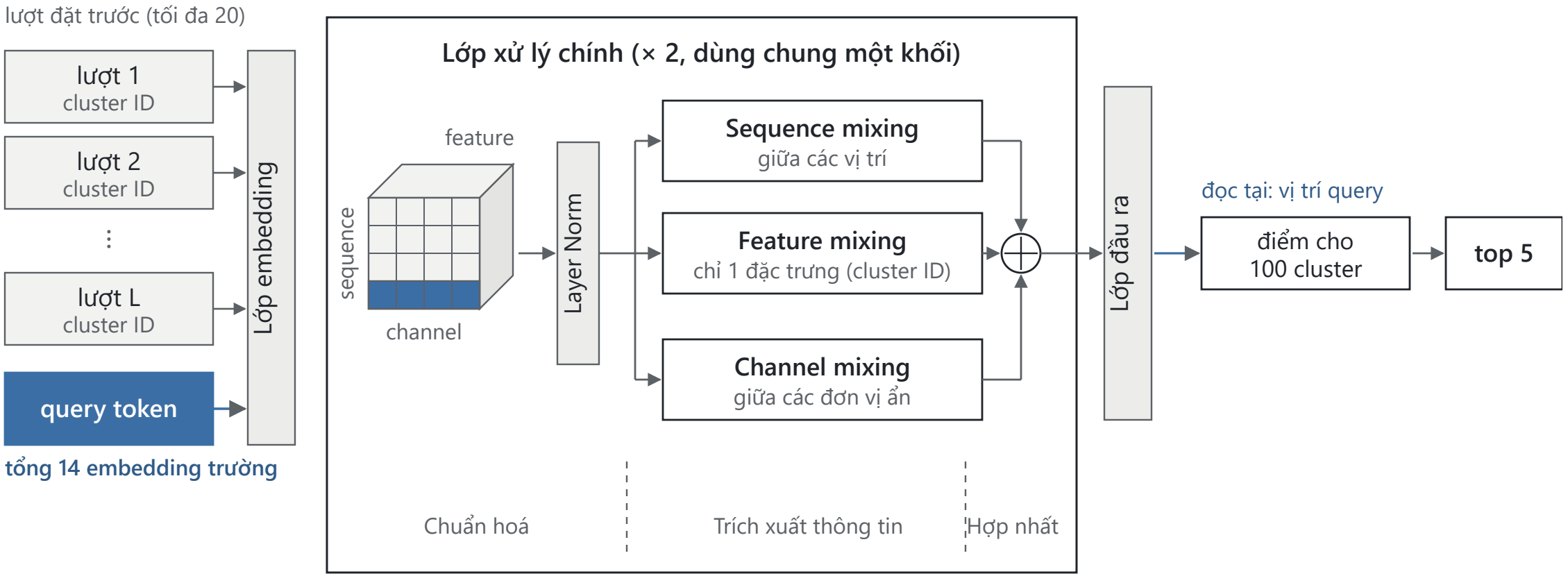
Mô hình	Recall@5	Recall@10	Recall@20	NDCG@5	NDCG@10	NDCG@20
ItemKNN (chỉ dùng lịch sử)	0.3036	0.4383	0.6097	0.2089	0.2522	0.2954
SMLP4Rec gốc (train chỉ trên dòng có lịch sử)	**0.3212**	**0.4530**	**0.6230**	**0.2274**	**0.2698**	**0.3127**
Logistic regression (điểm đến + ngữ cảnh tìm kiếm)	0.4969	0.6897	0.8578	0.3354	0.3978	0.4406

Giới hạn: SMLP4Rec gốc chỉ được train trên các lượt đặt mà khách đã có lịch sử trước đó (L ≥ 1) nên chỉ chấm được trên dòng warm; ItemKNN và logistic regression train trên mọi lượt đặt. Bảng chỉ so trên dòng warm của tập test, n = 213,869.

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/final_benchmark.csv (plain_warm, itemknn, logreg; dòng warm). Expedia, tập test, seed 2022.

Kết hợp hướng context-aware: query token đưa 14 trường tìm kiếm vào SMLP4Rec



[Phụ lục A1: Chi tiết về query token]

Nguồn: src/nbp/models/smlprec_query.py; results/week4_rebuild/smlprec_query_cold_train_es_run.json.

Thêm query token, SMLP4Rec vượt cả hai baseline trên toàn bộ lượt đặt

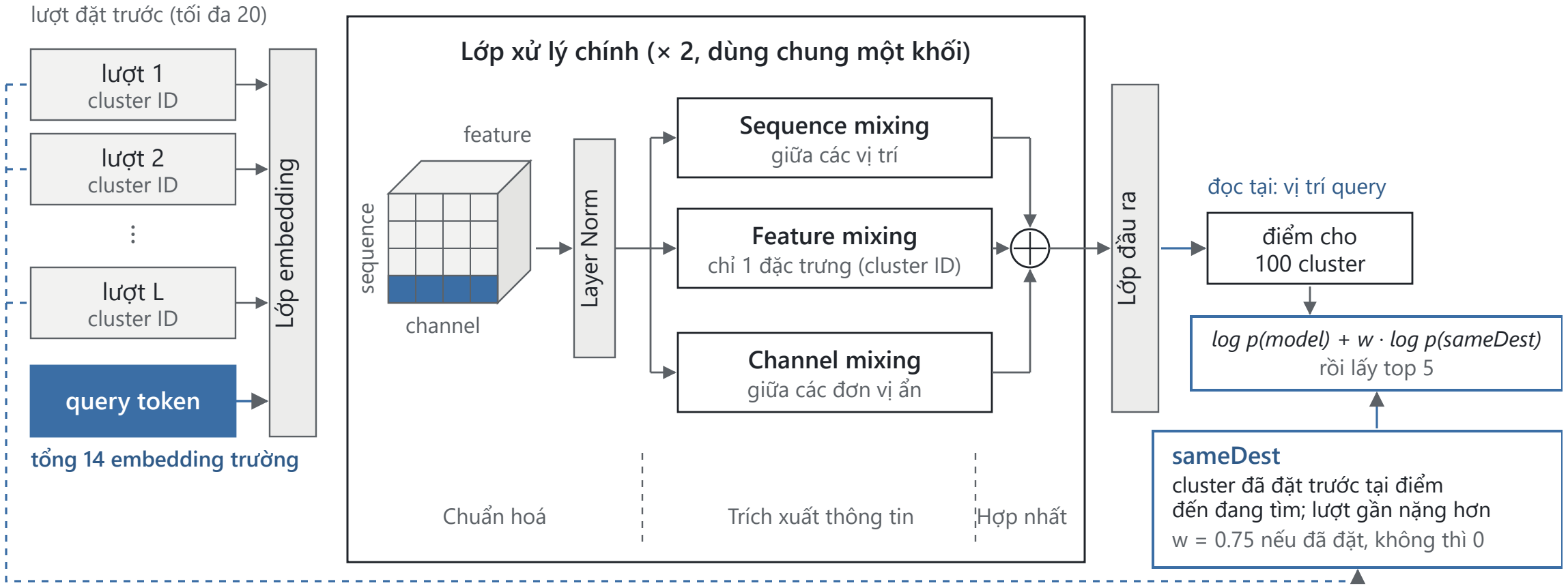
Mô hình	Recall@5	Recall@10	Recall@20	NDCG@5	NDCG@10	NDCG@20
ItemKNN (chỉ dùng lịch sử)	0.2713	0.3989	0.5626	0.1850	0.2260	0.2671
SMLP4Rec gốc (train chỉ trên dòng có lịch sử)	0.2856	0.4105	0.5734	0.2000	0.2401	0.2811
Logistic regression (điểm đến + ngữ cảnh tìm kiếm)	0.4948	0.6869	0.8560	0.3339	0.3961	0.4391
SMLP4Rec + query token (train mọi lượt đặt)	**0.5831**	**0.7613**	**0.9050**	**0.4122**	**0.4700**	**0.5066**

Bảng tính trên toàn bộ lượt đặt của tập test (warm và đầu tiên), n = 263,331. Với SMLP4Rec gốc, các lượt đặt đầu tiên không được mô hình chấm mà xếp theo độ phổ biến.

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/final_benchmark.csv (plain_warm, itemknn, logreg, query_token; dòng all). Expedia, tập test, seed 2022.

sameDest bổ sung những gì khách đã đặt trước đây tại điểm đến đang tìm



[Phụ lục A3: Chi tiết về sameDest]

Nguồn: src/nbp/hybrid/; results/week4_rebuild/smlprec_query_cold_train_es_run.json (samedest_hybrid).

Thêm sameDest, mô hình chính dẫn đầu mọi mô hình trên toàn bộ lượt đặt

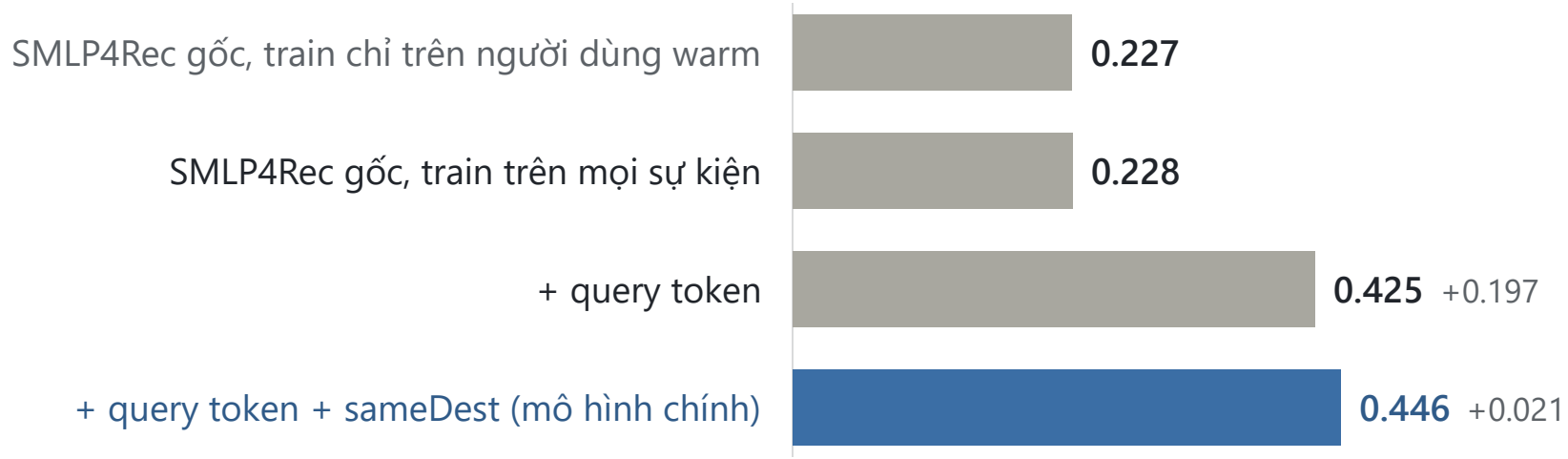
Mô hình	Recall@5	Recall@10	Recall@20	NDCG@5	NDCG@10	NDCG@20
ItemKNN (chỉ dùng lịch sử)	0.2713	0.3989	0.5626	0.1850	0.2260	0.2671
SMLP4Rec gốc (train chỉ trên dòng có lịch sử)	0.2856	0.4105	0.5734	0.2000	0.2401	0.2811
Logistic regression (điểm đến + ngữ cảnh tìm kiếm)	0.4948	0.6869	0.8560	0.3339	0.3961	0.4391
SMLP4Rec + query token	0.5831	0.7613	0.9050	0.4122	0.4700	0.5066
Mô hình chính (query token + sameDest)	**0.5944**	**0.7664**	**0.9066**	**0.4295**	**0.4853**	**0.5210**

Bảng tính trên toàn bộ lượt đặt của tập test (warm và đầu tiên), n = 263,331. Với SMLP4Rec gốc, các lượt đặt đầu tiên không được mô hình chấm mà xếp theo độ phổ biến.

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/final_benchmark.csv (plain_warm, itemknn, logreg, query_token, main; dòng all). Expedia, tập test, seed 2022.

query token mang phần lớn mức tăng, sameDest thêm một bước nhỏ hơn nhưng có thật



NDCG@5 trên dòng test warm. Các bước: query token +0.197 (KTC 95% +0.196 đến +0.199), sameDest +0.021 (KTC 95% +0.021 đến +0.022).

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/final_benchmark.csv, bootstrap_ci.csv. Expedia test, dòng warm, n = 213,869, seed 2022.

Mô hình chính dẫn trước cả hai baseline trên dòng test tất cả, cold và warm

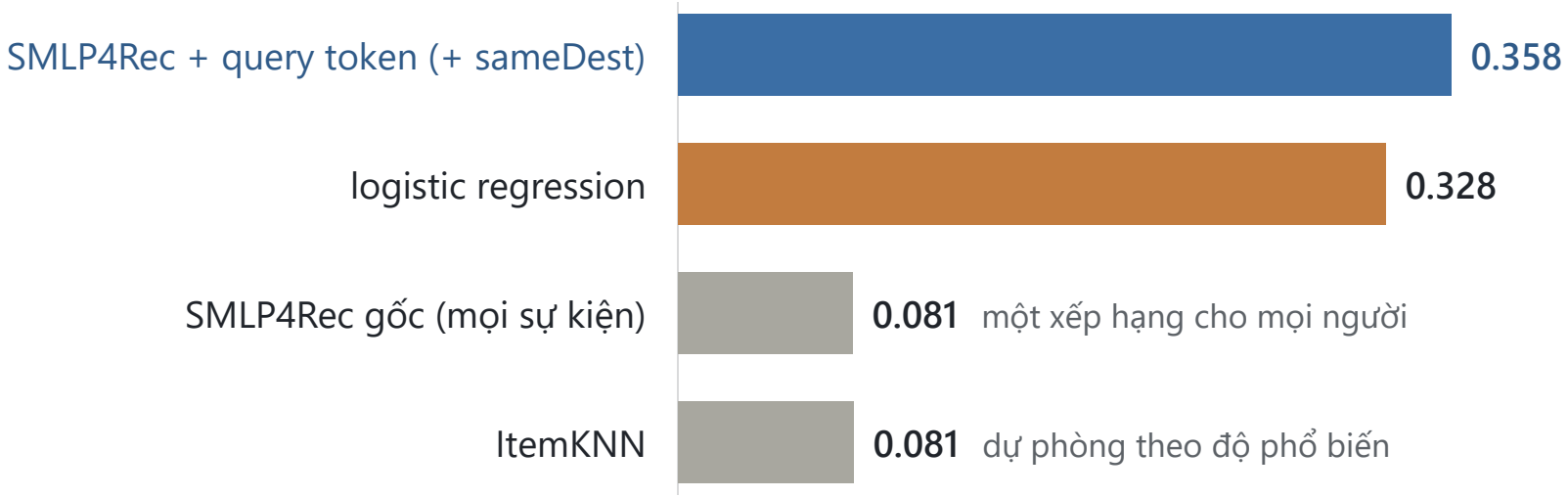
	R@5 tất cả	R@5 cold	R@5 warm	NDCG@5 tất cả	NDCG@5 cold	NDCG@5 warm
ItemKNN	0.271[a]	0.132[a]	0.304	0.185[a]	0.081[a]	0.209
SMLP4Rec gốc, train chỉ trên người dùng warm	0.286[a]	0.132[a]	0.321	0.200[a]	0.081[a]	0.227
SMLP4Rec gốc, train trên mọi sự kiện	0.286	0.131	0.321	0.200	0.081	0.228
logistic regression	0.495	0.486	0.497	0.334	0.328	0.335
+ query token	0.583	0.525	0.597	0.412	0.358	0.425
+ query token + sameDest (chính)	**0.594**	**0.525**	**0.610**	**0.429**	**0.358**	**0.446**

[a] Dự phòng theo độ phổ biến trên dòng lượt đặt đầu tiên, không phải điểm của mô hình. Chính trừ logistic regression, NDCG@5: warm +0.111 (KTC 95% +0.109 đến +0.112), cold +0.030 (KTC 95% +0.028 đến +0.033).

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/final_benchmark.csv, bootstrap_ci.csv. Expedia test: tất cả n = 263,331, cold n = 49,462, warm n = 213,869. Seed 2022.

Trên lượt đặt đầu tiên, chỉ các mô hình có xét tìm kiếm mới xếp hạng tốt

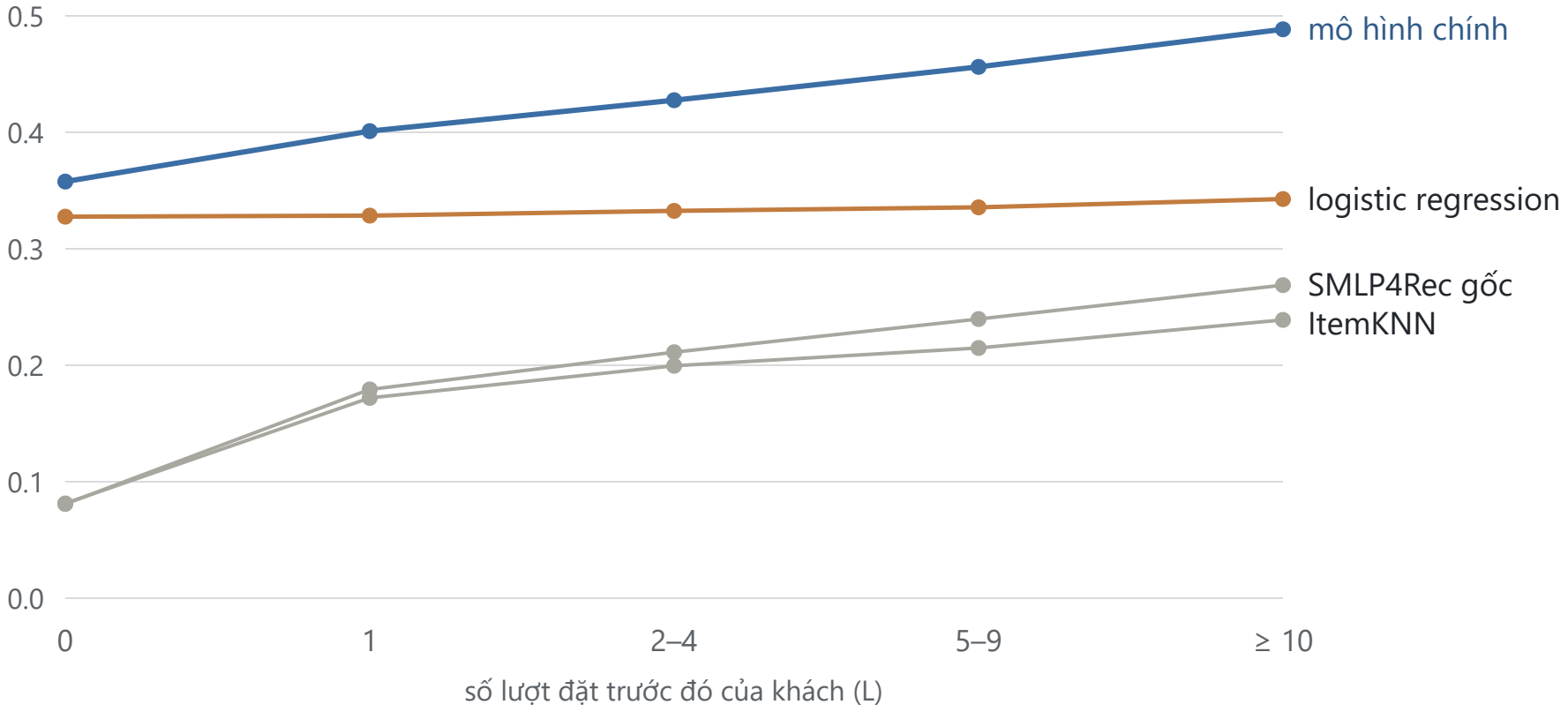


Mô hình chính dẫn trước logistic regression +0.030 (KTC 95% +0.028 đến +0.033) NDCG@5.

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/final_benchmark.csv, bootstrap_ci.csv. Expedia test, dòng lượt đặt đầu tiên, n = 49,462.

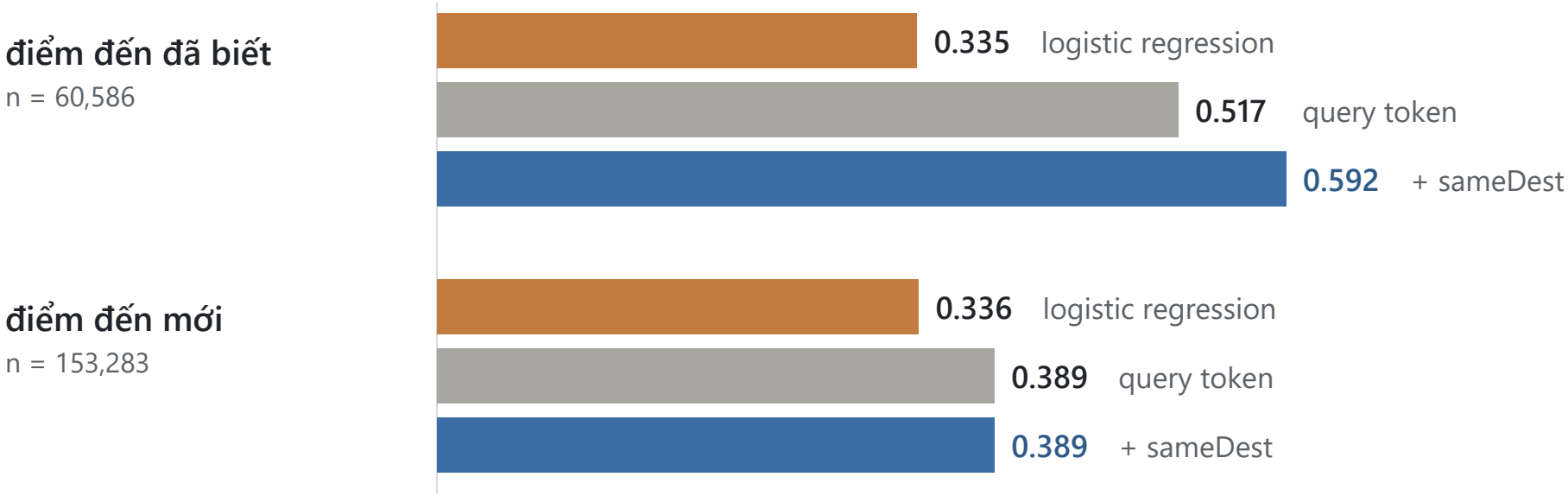
Khoảng dẫn trước logistic regression tăng theo lịch sử, từ +0.030 đến +0.146 NDCG@5



Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/slices_history_length.csv. Expedia test, mọi dòng, n = 263,331.

Lịch sử có giá trị ở nơi khách quay lại: sameDest nâng NDCG@5 tại điểm đến đã biết từ 0.517 lên 0.592



Loại các lượt đặt lại trong vòng một ngày, mức tăng của sameDest trên dòng warm vẫn là +0.012 NDCG@5.

Cách tính Recall@K, NDCG@K

Nguồn: results/week5_final/slices_destination_known.csv, bootstrap_ci.csv. Expedia test, dòng warm chia theo điểm đến.

Tìm kiếm hiện tại là tín hiệu chính, lịch sử là phần bổ sung nhỏ hơn nhưng có thật

- SMLP4Rec có xét tìm kiếm dẫn trước logistic regression +0.111 và ItemKNN +0.237 NDCG@5 trên dòng warm.
- Lượt đặt đầu tiên được chấm tốt, không cần lịch sử.
- sameDest giúp khách quay lại tại điểm đến đã biết.

Giới hạn: cluster chỉ là proxy cho phòng hoặc gói, một dataset, chỉ offline, không có dữ liệu trong thời gian lưu trú, hai baseline đơn giản, một seed.

Tiếp theo: thêm hai seed, chia rolling-origin để xét mùa vụ.

Nguồn: results/week5_final/final_benchmark.csv, bootstrap_ci.csv. Expedia test, dòng warm.

Phụ lục A4. Cấu hình chi tiết của các mô hình

Dữ liệu và đánh giá	chia theo thời gian 80/10/10 trên mục tiêu next-booking; lịch sử tối đa 20 lượt; xếp hạng đầy đủ 100 cluster; Recall và NDCG tại 5, 10, 20; chọn mô hình theo NDCG@10 trên valid; seed 2022
SMLP4Rec	2 lớp dùng chung một khối, hidden 64, expansion 3, dropout 0, GELU, loss cross-entropy; 33,702 tham số (bản gốc), 1,050,204 (có query token)
Query token	14 trường tìm kiếm; điểm đến và quốc gia người dùng cần ít nhất 5 lượt đặt ở train mới có embedding riêng, các trường khác ít nhất 1; 1,016,256 tham số của các embedding trường
Huấn luyện	Adam, learning rate 0.001, weight decay 0 (mặc định RecBole), batch 1024, tối đa 10 epoch trên CPU; early stopping theo NDCG@10 trên valid, patience 2, min_delta 0.0007; lần chạy chính chọn epoch 8, mất 3.1 giờ; bản SMLP4Rec gốc train chỉ trên dòng có lịch sử chạy cố định 3 epoch
sameDest	cơ số 0.7, $\alpha = 0.2$, $w = 0.75$ nếu điểm đến đã có trong lịch sử, 0 nếu không; lưới $\alpha \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$, $w \in \{0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 2, 2.5, 3\}$, chọn trên valid theo một sai số chuẩn
ItemKNN	cosine (thư viện implicit), $K = 50$ chọn từ {5, 10, 20, 50, 100} trên valid; đầu vào là 20 lượt đặt gần nhất; lượt đặt đầu tiên xếp theo độ phổ biến của các lượt đặt đầu tiên
Logistic regression	sklearn, saga, multinomial, $C = 1$ chọn từ {0.1, 1, 10}, dừng ở lượt lặp 8 theo valid; 14 trường tìm kiếm, không dùng lịch sử

Nguồn: configs/smlprec_expedia.yaml; results/week4_rebuild/ (smlprec_query_cold_train_es_run.json, smlprec_plain_cold_train_es_run.json, baselines_es_run.json).

Phụ lục A1. 14 trường của query token, đều thuộc lần tìm kiếm

Trường	Ý nghĩa	Cách mã hóa	Số giá trị
`dest`	Điểm đến đang tìm (srch_destination_id)	id gốc; điểm đến có dưới 5 lượt đặt ở train thành "ngoài từ vựng"	15,509
`dest_type`	Loại điểm đến	id gốc	9
`ci_month`	Tháng check-in	1 đến 12; 0 = không rõ	12
`lead`	Số ngày từ lúc tìm đến check-in	10 khoảng, từ 0 ngày đến từ 365 ngày; 0 = không rõ	11
`stay`	Số đêm lưu trú	0 đến 14 đêm, dài hơn gộp chung; 0 = không rõ	14
`adults`	Số người lớn	cắt ở 9; 0 = không rõ	10
`children`	Số trẻ em	cắt ở 9	10
`rooms`	Số phòng	cắt ở 8; 0 = không rõ	9
`package`	Đặt kèm gói (is_package)	0 hoặc 1	2
`mobile`	Tìm trên thiết bị di động	0 hoặc 1	2
`channel`	Kênh tiếp thị	id gốc	11
`site`	Website Expedia (site_name)	id gốc	41
`posa`	Châu lục điểm bán	id gốc	5
`country`	Quốc gia của người dùng	id gốc; dưới 5 lượt đặt ở train thành "ngoài từ vựng"	219

← Quay lại slide 10

Nguồn: src/nbp/data/query_features.py; smlprec_query_cold_train_es_run.json (query_coverage). Số giá trị = kích thước từ vựng, tính trên train.

Phụ lục A2. Recall@K và NDCG@K tính từ thứ hạng của cluster đúng

Mỗi lượt đặt có đúng một cluster đúng. Mô hình chấm điểm cả 100 cluster, không lấy mẫu negative, rồi xếp hạng; gọi `r` là thứ hạng của cluster đúng (1 là tốt nhất). Đồng điểm thì cluster có chỉ số cột nhỏ hơn xếp trước.

`Recall@K = 1[r ≤ K]`

`NDCG@K = 1[r ≤ K] / log2(r + 1)` (DCG lý tưởng bằng 1 vì chỉ có một cluster đúng)

Kết quả là trung bình trên các dòng đang xét, với K = 5, 10, 20. Ví dụ bốn lượt đặt có `r` = 1, 3, 6, 11 và K = 5:

r	Recall@5	NDCG@5
1	1	1.000
3	1	0.500
6	0	0.000
11	0	0.000
trung bình	**0.500**	**0.375**

← Quay lại

Nguồn: src/nbp/eval/metrics.py (target_rank, recall_at_k, ndcg_at_k, summarize); ví dụ khớp tests/test_metrics.py.

Phụ lục A3. Công thức sameDest và điểm cuối cùng

Động lực: trên dòng warm của test, 29.0% tìm kiếm diễn ra tại điểm đến khách từng đặt; trong số đó 51.6% lại đặt đúng một cluster đã đặt tại điểm đó (14.96% toàn bộ dòng warm).

`sameDest(k) = Σ 0.7^tuổi` trên các lượt đặt trước tại điểm đến đang tìm có cluster k, rồi chuẩn hóa: `q_sameDest(k) = sameDest(k) / Σj sameDest(j)`

`điểm(k) = log q_mô hình(k) + w · log q_sameDest(k)`, với `log q = log((1 − α)·p + α/100)`, α = 0.2; w = 0.75 nếu điểm đến đã có trong lịch sử, w = 0 nếu không.

Ví dụ: lịch sử bốn lượt đặt, tìm kiếm tại điểm đến A.

Điểm đến	Cluster	Tuổi	Tính vào sameDest?	0.7^tuổi
A	7	3	có	0.343
B	9	2	không (khác điểm đến)	0
A	12	1	có	0.700
A	7	0	có	1.000

Tìm kiếm tại điểm đến A: cluster 7 có 0.343 + 1.000 = 1.343, cluster 12 có 0.700; sau chuẩn hóa `q_sameDest`(7) = 0.657, `q_sameDest`(12) = 0.343.

Cơ số 0.7 là giá trị đặt cố định, không tune: $0.7^2 \approx 0.5$ nên cứ hai lượt đặt cũ hơn thì trọng số còn khoảng một nửa. Tuổi 0 là lượt gần nhất.

← Quay lại

Nguồn: src/nbp/hybrid/same_dest.py, fusion.py; eda_clean.json (E5); smlprec_query_cold_train_es_run.json (samedest_hybrid). Ví dụ tính bằng same_dest_scores.